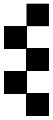
NoSQL Implementing MongoDB in Python Applications



University of Essex

NoSQL

- NoSQL
  - Not only SQL

- NoSQL focus on
  - semi-structured data storage
  - high performance
  - Availability
  - data replication
  - scalability

NoSQL Databases

- MongoDB and CouchDB
  - Document stores
- Neo4J and GraphBase
  - Graph-based NOSQL systems
- OrientDB
  - Combines several concepts
- Database systems classified on the object model
  - Or native XML model

Python MongoDB Driver

- The **Python MongoDB driver** is a library (**pymongo**) that provides the necessary objects.

- These **objects** represent the connection, database, collection cursor, and documents on the MongoDB server and provide the necessary functionality to **integrate** data from a **MongoDB database** into your **Python applications**.

Python MongoClient Object

- The **Python MongoClient object** provides the functionality to **connect** to the **MongoDB server** and **access databases**.
- The **first step** you take in implementing MongoDB in your Python applications is to **create an instance** of the MongoClient object.
- To create an instance of the MongoClient object, you need to use MongoClient() with the appropriate options. The most basic form connects on the **localhost** with the **default port**.
- For example, client = MongoClient()

Python Database Object

- The **Python Database object** provides the functionality to **access and manipulate collections**.
- The databases associated with a MongoClient are stored as part of the MongoClient object's internal dictionary.
- The simplest method of getting an instance of a Database object is to access it directly by name on the MongoClient object.
- For example, db = client.test

Python Collection Object

- The **Python Collection object** provides the functionality to **access and manipulate documents** in a collection.
- The collections associated with a Database are stored as part of the Database object's internal dictionary.
- The simplest method of getting an instance of a Collection object is to access it directly by name on the Database object.
- For example, db.restaurants
- After you have created an instance of the Collection object, you can use the object to access the collection.

Python Cursor Object

- The **Python Cursor object** represents **a set of documents** on the MongoDB server.
- Typically, a Cursor object is returned when you query the collection using a find operation.
- Instead of returning the full set of document objects to the Python application, a Cursor object is returned, enabling you to access the documents in a controlled manner from Python.

For example

- cursor = db.restaurants.find()
- cursor = db.restaurants.find({"borough": "Manhattan"})

Sample Dataset

The sample_restaurants database contains the following collections:

Collection Name	Description
restaurants	Contains details on restaurants.
neighborhoods	Contains details on neighborhoods.

A Document from collection


```
{
"address": {
"building": "1007",
"coord": [ -73.856077, 40.848447 ],
"street": "Morris Park Ave",
"zipcode": "10462"
},
"borough": "Bronx",
"cuisine": "Bakery",
"grades": [
{ "date": { "$date": 1393804800000 }, "grade": "A", "score": 2 },
{ "date": { "$date": 1378857600000 }, "grade": "A", "score": 6 },
{ "date": { "$date": 1358985600000 }, "grade": "A", "score": 10 },
{ "date": { "$date": 1322006400000 }, "grade": "A", "score": 9 },
{ "date": { "$date": 1299715200000 }, "grade": "B", "score": 14 }
],
"name": "Morris Park Bake Shop",
"restaurant_id": "30075445"
}
```

JavaScript Object Notation (JSON)

- JSON is a language-independent data format.
- It was derived from JavaScript, but many modern programming languages include code to generate and parse JSON-format data.
- JSON is a lightweight format for storing and transporting data
- JSON is "self-describing" and easy to understand. For more details, go to What is JSON (w3schools.com)

JSON

JSON uses human-readable text to store and transmit data objects consisting of attribute–value pairs and array data types (or any other serializable value).

Overview of JSON file:


```
{"address": {"building": "1007", "coord": [-73.856077, 40.848447], "street": "Morris Park Ave", "zipcode": "10462"}, "borough
{"address": {"building": "469", "coord": [-73.961704, 40.662942], "street": "Flatbush Avenue", "zipcode": "11225"}, "borough"
{"address": {"building": "351", "coord": [-73.98513559999999, 40.7676919], "street": "West   57 Street", "zipcode": "10019"},
{"address": {"building": "2780", "coord": [-73.98241999999999, 40.579505], "street": "Stillwell Avenue", "zipcode": "11224"},
{"address": {"building": "97-22", "coord": [-73.8601152, 40.7311739], "street": "63 Road", "zipcode": "11374"}, "borough": "Q
{"address": {"building": "8825", "coord": [-73.8803827, 40.7643124], "street": "Astoria Boulevard", "zipcode": "11369"}, "bor
{"address": {"building": "2206", "coord": [-74.1377286, 40.6119572], "street": "Victory Boulevard", "zipcode": "10314"}, "bor
{"address": {"building": "7114", "coord": [-73.9068506, 40.6199034], "street": "Avenue U", "zipcode": "11234"}, "borough": "B
{"address": {"building": "6409", "coord": [-74.00528899999999, 40.628886], "street": "11 Avenue", "zipcode": "11219"}, "borou
{"address": {"building": "1839", "coord": [-73.9482609, 40.6408271], "street": "Nostrand Avenue", "zipcode": "11226"}, "borou
{"address": {"building": "2300", "coord": [-73.8786113, 40.8502883], "street": "Southern Boulevard", "zipcode": "10460"}, "bo
{"address": {"building": "7715", "coord": [-73.9973325, 40.61174889999999], "street": "18 Avenue", "zipcode": "11214"}, "boro
{"address": {"building": "1269", "coord": [-73.871194, 40.6730975], "street": "Sutter Avenue", "zipcode": "11208"}, "borough"
{"address": {"building": "1", "coord": [-73.96926909999999, 40.7685235], "street": "East   66 Street", "zipcode": "10065"}, "
{"address": {"building": "705", "coord": [-73.9653967, 40.6064339], "street": "Kings Highway", "zipcode": "11223"}, "borough"
{"address": {"building": "203", "coord": [-73.97822040000001, 40.6435254], "street": "Church Avenue", "zipcode": "11218"}, "b
{"address": {"building": "265-15", "coord": [-73.7032601, 40.7386417], "street": "Hillside Avenue", "zipcode": "11004"}, "bor
{"address": {"building": "6909", "coord": [-74.0259567, 40.6353674], "street": "3 Avenue", "zipcode": "11209"}, "borough": "B
{"address": {"building": "522", "coord": [-73.95171, 40.767461], "street": "East   74 Street", "zipcode": "10021"}, "borough"
{"address": {"building": "284", "coord": [-73.9829239, 40.6580753], "street": "Prospect Park West", "zipcode": "11215"}, "bor
{"address": {"building": "129-08", "coord": [-73.839297, 40.78147], "street": "20 Avenue", "zipcode": "11356"}, "borough": "Q
{"address": {"building": "759", "coord": [-73.9925306, 40.7309346], "street": "Broadway", "zipcode": "10003"}, "borough": "Ma
{"address": {"building": "3406", "coord": [-73.94024739999999, 40.7623288], "street": "10 Street", "zipcode": "11106"}, "boro
{"address": {"building": "502", "coord": [-73.976112, 40.786714], "street": "Amsterdam Avenue", "zipcode": "10024"}, "borough
{"address": {"building": "730", "coord": [-73.96805719999999, 40.7925587], "street": "Columbus Avenue", "zipcode": "10025"},
```